Scope

I compared 001_initial_schema.sql with the files in the current workspace. The project currently contains a browser-only prototype: I found no backend routes/controllers, models, validation layer, API requests/responses, or SQL queries to compare. Its data is primarily sample data stored in localStorage through data.js and script.js.

I inspected the SQL files but did not execute SQL or modify files.

1. Fields/features used by the code but missing from the schema

Profile preferences: The student profile stores emailVisibility, timezone, and notification preferences. I don't see corresponding fields or a preferences table. A profile picture field exists, but its storage format needs attention below.

Official document requests: The UI tracks requests for COR, COE, and TOR, including whether they were released. The schema has document types and uploaded student documents, but no distinct request/release record with requester, status, and timestamps.

Academic rules: The frontend uses passing-grade and Dean's List thresholds, plus attendance thresholds and excused-absence rules. grade_scales captures some grade information, but the schema doesn't clearly represent those policies.

Application workflow state: The UI tracks draft/enrollment-step completion and confirmation. Some of this may be transient UI state, but if it must survive sessions or be audited, it needs explicit storage.

Staff office: The registrar settings UI includes an office field; the schema has department office_location, but no clear individual staff office field.

2. Schema fields/features not currently used by the code

The schema is much broader than this prototype. The current UI does not use or persist most of the data for:

MFA challenges, password resets, lockouts, and account lifecycle.

Educational background and course prerequisites.

Assessment definitions and raw assessment scores.

Detailed schedule conflicts and faculty assignments.

Attendance session records, alert rules, and delivery status.

Report definitions/history, backup configuration/history, retention policies, and performance metrics.

Some related screens exist, but show hard-coded/sample information rather than reading or writing these tables. So these are currently future-facing schema features, not active integrations. The profile-registration UI also exposes fields such as date of birth and guardian/emergency contacts, but the current registrar save handler does not persist all of them.

3. Naming and data-type mismatches

Grade model: The frontend has midterm, finals, an overall finalGrade, and a separate status for each grade period. student_grades has preliminary_grade, midterm_grade, final_grade, one optional grading_period_id, and one grade_status. That mixes per-period scores with an overall grade and does not directly represent the frontend model. See the sample grade shape and the grade table.

Grade scale units: The frontend uses numeric grades on a 1–5 scale, while grade_scales.minimum_percentage and maximum_percentage read as percentage ranges. Decide whether the stored values are grade equivalents or percentages, and model/label them consistently.

Statuses: The UI uses lowercase values such as pending, review, and posted; the schema defaults include title-case values such as Pending and Draft. If the API returns these values unchanged, frontend status checks may not match. Choose canonical values or normalize them in the API.

Image storage: The UI can hold an uploaded image as a data URI, up to 5 MB, while students.profile_picture is only a VARCHAR(500). Store a private object-storage key/URL instead of a base64 image in that column.

IDs and terms: The prototype uses string IDs like 2627-1 and subject slugs, while the schema uses integer primary keys and separate business codes. The API will need to map these identifiers explicitly.

Prerequisite grade: course_prerequisites.minimum_grade is text even though the academic grade is numeric; use a numeric type if this is meant to be compared with grades.

4. Missing or incorrect relationships

The foreign keys generally cover the main entities, but several permit inconsistent records:

class_enrollments.student_id can differ from the student on its referenced student_enrollments record.

attendance_records can reference a student, class enrollment, and attendance session that do not belong together.

student_grades can have a student, enrollment, and term that don't match.

assessment_scores does not ensure the student is enrolled in the class offering for the assessment.

A profile-correction request can reference a supporting document owned by a different student.

admission_applications.student_id can point to a student whose person differs from the application's person_id.

Where practical, enforce these with composite foreign keys/unique keys, or derive redundant student/term IDs through the enrollment relationship. The supporting-document relationship should also verify ownership.

The separate guardians and emergency_contacts tables overlap: guardians can be marked emergency contacts, while emergency contacts are modeled separately. Choose a clear source of truth to avoid duplicated or conflicting contact records.

5. Missing indexes and unique constraints

user_roles has a composite unique key containing nullable department_id and effective_from. MySQL/MariaDB unique indexes permit multiple rows with NULL in those columns, so duplicate assignments can still be inserted. Use a design that enforces uniqueness for both global and department-scoped assignments.

student_grades lacks a unique constraint for the intended grade grain—for example, one record per class enrollment and grading period. Duplicate grade rows could be created.

is_current_term is indexed, but nothing prevents multiple terms from being marked current. Enforce at most one current term.

The schema does not enforce one primary guardian per student.

The UI prevents duplicate applications in client-side code, but the database has no corresponding protection against concurrent duplicate active applications for the same person, program, and term.

As there are no database queries in the project yet, query-specific indexes cannot be responsibly assessed. The migration already indexes many foreign keys and common status fields.

6. Existing tables/migrations that conflict

I found no SQL migrations in the application workspace. The database bundle contains 001_initial_schema.sql and a separate seed.sql; the seed file inserts role and permission rows rather than redefining tables, so it does not directly conflict with the migration.

There is, however, a configuration mismatch: the seed defines ICT and EdTech roles, but does not create department records or assign permissions to those roles. The UI's user list is separate sample data, so it does not yet correspond to those seeded roles. Also, the initial migration is not designed to be rerun against an existing database; apply it only once to a new database or through a migration runner that records applied migrations.

7. Security concerns and recommendations

Authentication is not implemented. The current login checks only that fields are filled, and the admin area is selected based on the username prefix—not a verified password or role. See the login handler and the admin check. The users table provides a place for password hashes, but there is no server enforcing authentication.

Authorization must be server-side. The frontend exposes data in browser storage, and SIMSData.get() returns the full academic data object, including unposted grades. Hiding UI elements or using client-side role checks cannot protect student records.

Fail closed on permission changes. user_roles.department_id uses ON DELETE SET NULL; if NULL means global access, deleting a department could broaden a user's scope. Similarly, report_definitions.required_permission_code becomes NULL when its

permission is deleted. Restrict permission/dept deletion or ensure authorization treats missing scope/permission as denied.

Protect student documents. Validate file type and size on the server, store files outside public web paths, and authorize every download. Client-side checks alone are not security controls.

Add server-side validation and constraints for grade ranges, score bounds, date ordering, valid status values, and application/enrollment consistency. Use parameterized database queries in the future backend.

Priority before connecting a live database: implement server-side authentication and role/scope authorization; settle the grade table design; add relationship/uniqueness constraints for enrollments and grades; and define document-request and profile-preference storage.